

Credit Card Default Prediction Using Machine Learning (UCI Dataset)

1. Introduction

Credit card default prediction is a **binary classification problem**, where the objective is to predict whether a customer will fail to pay their credit card bill in the next month.

This problem is highly relevant for banks and financial institutions because **accurately predicting defaults** allows them to:

- Minimize financial losses
- Adjust credit limits
- Prioritize monitoring of high-risk customers

Traditional credit evaluation methods rely on **fixed rules** or **simple statistical methods**. While useful, these approaches:

- Do not scale well for large datasets
- Cannot capture complex patterns in customer repayment behavior

Machine learning techniques allow the model to learn relationships between:

- Demographic features
- Financial behavior
- Repayment history

...and use these patterns to make accurate predictions.

2. Data Description

The dataset used in this project is the **UCI Credit Card Default Dataset**, which contains information about **30,000 credit card customers in Taiwan**.

Target Variable

`default.payment.next.month`

Value	Meaning
1	Customer defaulted
0	Customer did not default

Note: The target variable is **imbalanced**, with more non-default customers than defaults.

Features

The dataset contains **24 predictive features** (after dropping the ID column).

Demographic Features

- **SEX:** Gender (1 = male, 2 = female)
- **EDUCATION:** Education level (1 = graduate school, 2 = university, 3 = high school, 4 = others)
- **MARRIAGE:** Marital status (1 = married, 2 = single, 3 = others)
- **AGE:** Age of the customer

Credit Features

- **LIMIT_BAL:** Total credit limit

Repayment Status Features

- **PAY_0 to PAY_6:** Repayment status for the past 6 months
 - 0 = on-time repayment
 - Positive integers = months delayed
 - Higher values indicate higher default risk

Bill Amount Features

- **BILL_AMT1 to BILL_AMT6:** Monthly statement amounts for the past 6 months

Payment Amount Features

- **PAY_AMT1 to PAY_AMT6:** Amount paid for the past 6 months

3. Environment Setup

Before starting any machine learning project, it is essential to set up the environment with the necessary libraries and configurations. This ensures smooth execution, reproducibility, and meaningful analysis of results.

Libraries and Their Purpose:

- **Warnings:**
 - We suppress irrelevant warnings to make the output cleaner and more readable. Warnings can be helpful for debugging, but many are unnecessary and can clutter the console.
- **NumPy:**
 - NumPy is a fundamental library for numerical computation in Python.
 - It is used for efficient array operations and mathematical calculations.
 - Setting a seed in NumPy ensures reproducibility for random operations such as data shuffling or random number generation.
- **pandas:**
 - pandas is used for data manipulation and analysis.
 - It provides powerful structures like DataFrames and Series to handle tabular data efficiently.
 - Common operations include data cleaning, filtering, merging, and aggregation.
- **Matplotlib & Seaborn:**
 - These libraries are used for data visualization.
 - **Matplotlib** provides detailed plotting functionalities, while **Seaborn** simplifies the creation of attractive statistical plots.
 - They are particularly useful for visualizing confusion matrices, feature distributions, and performance comparisons of different models.
- **scikit-learn (sklearn):**
 - A comprehensive library for machine learning.
 - It provides tools for model training, evaluation, preprocessing, and splitting datasets into training and testing sets.
 - Functions like `train_test_split`, `StandardScaler`, and performance metrics are included here.
- **imblearn:**

- Used for handling class imbalance in datasets.
- The SMOTE (Synthetic Minority Over-sampling Technique) method generates synthetic samples for minority classes, helping models learn better from imbalanced data.

Reproducibility:

Ensuring reproducibility is critical in machine learning to obtain consistent results across runs. This is achieved by setting a fixed random seed:

```
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

4. Data Cleaning

Data cleaning is a crucial step in any machine learning project. It ensures that the dataset is accurate, consistent, and suitable for modeling. One important aspect of cleaning is correcting **categorical variables** that contain invalid or unexpected values.

Categorical Correction:

- **EDUCATION:**
 - Original dataset contained invalid values such as 0, 4, 5, 6.
 - These values were **grouped into a new category 4** to represent “other” or “unknown” education levels.
 - **Reason:** Combining these rare or invalid values prevents the creation of sparse categories, which could otherwise reduce the reliability of the model. Sparse categories can lead to poor model generalization, as the model sees very few examples of those categories during training.
- **MARRIAGE:**
 - The dataset had an invalid value 0, which does not correspond to any meaningful marital status.
 - This value was **reassigned to category 3**, representing “others” or “unknown”.
 - **Reason:** Ensures that all entries in the marital status column are meaningful and allows the model to process the variable consistently.

By performing these corrections, we ensure that **categorical variables are clean, interpretable, and properly formatted**, which improves the **reliability and performance of machine learning models**.

5. Feature Engineering

Feature engineering is the process of transforming raw data into meaningful features that better represent the underlying patterns, trends, and risk factors in the dataset. Carefully engineered features often **improve model performance** by capturing domain-specific insights that raw variables alone may not reveal.

Key Engineered Features:

- **TOTAL_BILL_AMT:**
 - Defined as the sum of BILL_AMT1 through BILL_AMT6.
 - Represents the **total spending** of a customer over the past six months.
 - **Purpose:** Captures overall financial exposure and spending behavior.
- **TOTAL_PAY_AMT:**
 - Sum of PAY_AMT1 through PAY_AMT6.
 - Reflects the customer's **total repayment capacity**.
 - **Purpose:** Indicates how much the customer has actually paid towards their debts.
- **PAY_RATIO:**
 - Calculated as $\text{TOTAL_PAY_AMT} / (\text{TOTAL_BILL_AMT} + 1)$.
 - Represents the **ratio of repayment to billed amount**.
 - Values close to 1 suggest **responsible repayment behavior**, while low values indicate underpayment or potential risk.
- **BILL_TREND:**
 - Calculated as $\text{BILL_AMT1} - \text{BILL_AMT6}$.
 - Captures whether spending is **increasing or decreasing over time**.
 - Helps identify customers with rising debt trends that may signal risk.
- **AVG_PAY_DELAY:**
 - Mean of PAY_0 through PAY_6 (payment delays).
 - Represents the **average repayment delay**.
 - **Significance:** A strong predictor of default risk; higher delays indicate higher likelihood of default.
- **DELAY_TREND:**
 - Calculated as $\text{PAY_0} - \text{PAY_6}$.
 - Detects whether repayment behavior is **improving or worsening over time**.
 - Helps track customers' behavioral trends in debt repayment.
- **PAY_VOLATILITY:**
 - Standard deviation of PAY_AMT1 through PAY_AMT6.
 - Measures the **stability of payments**.

- High volatility may indicate inconsistent financial behavior or irregular income.
- **UTILIZATION:**
 - Calculated as $\text{BILL_AMT1} / (\text{LIMIT_BAL} + 1)$.
 - Indicates how much of the available credit limit is being used.
 - High utilization is a signal of **financial stress** and potential risk.

By engineering these features, we **capture both behavior patterns and financial health indicators**, providing the model with richer information to predict credit risk accurately.

6. Train-Test Split

Splitting the dataset into training and testing sets is a crucial step in machine learning. It allows the model to **learn patterns from the training set** and then **evaluate its performance on unseen data** using the testing set. This helps prevent overfitting and ensures that the model generalizes well to new data.

Setup:

- **Features (X):**
 - Includes all columns **except the target variable** (`default.payment.next.month`).
 - These are the input variables that the model uses to learn patterns.
- **Target (y):**
 - The target variable is `default.payment.next.month`.
 - It indicates whether a customer will default next month (usually represented as 0 = no default, 1 = default).
- **Split Ratio:**
 - The dataset is divided into **80% training** and **20% testing**.
 - The training set is used to fit the model, and the testing set is used to evaluate its predictive performance.
- **Stratified Split:**
 - The split is stratified, meaning the **proportion of each class (default vs. non-default) is preserved** in both training and testing sets.
 - This is important in imbalanced datasets to ensure the model sees a representative distribution of classes during training.
- **Random State = 42:**
 - Setting a fixed random state ensures **reproducibility**.
 - Every time the code is executed, the train-test split will remain identical, allowing consistent results for model evaluation.

By carefully splitting the dataset, we **create a fair and representative testing environment** for evaluating model performance while maintaining reproducibility.

7. Modeling Pipeline

A well-designed **modeling pipeline** ensures that all models are trained consistently, with the same preprocessing steps applied to the data. This improves reproducibility, reduces bias, and allows fair comparison between different models.

Pipeline Steps:

1. **RobustScaler:**
 - a. Scales features using the **median** and **interquartile range (IQR)** instead of the mean and standard deviation.
 - b. **Advantage:** Resistant to outliers, which are common in financial datasets.
 - c. Ensures that features are on a similar scale, which is important for many machine learning algorithms.
2. **PCA (Principal Component Analysis, 15 components):**
 - a. Reduces the dimensionality of the dataset while retaining the most important variance.
 - b. **Benefits:**
 - i. Removes multicollinearity between features.
 - ii. Reduces computational cost, making training faster.
 - iii. Simplifies the feature space, which can improve model generalization.
3. **SMOTE (Synthetic Minority Over-sampling Technique):**
 - a. Balances the training set by generating synthetic samples for the **minority class**.
 - b. **Purpose:** Prevents the model from being biased toward the majority class, which is especially important in imbalanced datasets like credit default prediction.
4. **Model (Estimator):**
 - a. The final step in the pipeline is the **machine learning estimator**, which can be:
 - i. Logistic Regression
 - ii. Random Forest
 - iii. Gradient Boosting
 - iv. K-Nearest Neighbors (KNN)
 - v. Naive Bayes
 - b. Each model uses the same preprocessed and balanced dataset, allowing for fair performance comparison.

8. Prediction and Threshold

After training a machine learning model, predictions are usually returned as **probabilities** rather than direct class labels. For example, a model may output a probability `y_prob` that a customer will default next month. To convert these probabilities into binary predictions (default vs. no default), a **decision threshold** is applied.

Threshold Setting:

- **Threshold = 0.45:**
 - If `y_prob ≥ 0.45`, the customer is predicted to default (`y_pred = 1`).
 - If `y_prob < 0.45`, the customer is predicted as non-default (`y_pred = 0`).
 - **Reasoning:** The threshold is slightly lower than the typical 0.5 to **increase recall**, ensuring that more potential defaulters are detected.

Benefits of Adjusting the Threshold:

- **Improves Recall:**
 - Detects more actual defaulters, which is critical in financial applications where missing a defaulter can be costly.
- **Balances Financial Risk vs. False Alarms:**
 - A lower threshold may increase false positives (non-defaulters predicted as defaulters), but this is often acceptable compared to missing real defaulters.
 - This balance ensures that the model aligns with financial risk management priorities.

Implementation:

```
y_pred = 1 if y_prob >= 0.45 else 0
```

9. Evaluation Metrics

In any predictive modeling task, especially in classification problems like credit default prediction, evaluation metrics are essential to measure how well the model performs. Here are the main metrics used:

1. Accuracy

- a. **Definition:** Accuracy is the proportion of total predictions that the model got correct.
- b. **Formula**
$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$
- c. **Interpretation:** While accuracy gives a general idea of model performance, it can be misleading if the dataset is imbalanced (e.g., if defaulters are much fewer than non-defaulters).

2. Precision

- a. **Definition:** Precision measures the proportion of predicted positive cases (e.g., predicted defaulters) that are actually positive.
- b. **Formula:**

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

- c. **Interpretation:** A high precision means that when the model predicts someone will default, it is usually correct. This is important when false alarms are costly.

3. Recall (Sensitivity)

- a. **Definition:** Recall measures the proportion of actual positive cases (e.g., actual defaulters) that the model correctly identifies.
- b. **Formula:**
$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$
- c. **Interpretation:** Recall is especially critical in credit risk models because missing actual defaulters can lead to significant financial losses. This is why recall is often considered the most important metric in default prediction.

4. F1-Score

- a. **Definition:** The F1-score is the harmonic mean of precision and recall.
- b. **Formula:**

$$\text{F1} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

- c. **Interpretation:** The F1-score balances precision and recall. It is often used to rank models on a leaderboard because it provides a single metric reflecting both correctness and completeness of positive predictions.

5. ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

- a. **Definition:** ROC-AUC measures the model's ability to rank predictions correctly across different threshold values.
- b. **Interpretation:** A higher ROC-AUC indicates better separation between defaulters and non-defaulters, even if the exact threshold is not fixed.

6. Confusion Matrix

- a. **Definition:** A confusion matrix is a table that visualizes model performance by showing:
 - i. **True Positives (TP):** Correctly predicted defaulters
 - ii. **True Negatives (TN):** Correctly predicted non-defaulters
 - iii. **False Positives (FP):** Non-defaulters incorrectly predicted as defaulters
 - iv. **False Negatives (FN):** Defaulters incorrectly predicted as non-defaulters
- b. **Interpretation:** It provides an intuitive way to see the types of errors the model makes, which is helpful for improving model strategies.

Summary: These metrics together provide a complete understanding of model performance. For imbalanced datasets, metrics like recall, F1-score, and ROC-AUC are more informative than accuracy alone.

10. Model Evaluation (Detailed Explanation)

After building predictive models, the next critical step is **evaluating their performance** to understand which model performs best in terms of predicting defaulters while minimizing errors. The evaluation is done systematically as follows:

1. Individual Model Evaluation

- a. Each model is tested independently using the **pre-defined evaluation pipeline**. This ensures consistency in how predictions are generated, metrics are computed, and results are compared.
- b. For each model, we calculate a set of performance metrics:
 - i. **Accuracy** – overall correctness of predictions
 - ii. **Precision** – proportion of predicted defaulters that actually defaulted
 - iii. **Recall** – proportion of actual defaulters correctly predicted
 - iv. **F1-Score** – harmonic mean of precision and recall
 - v. **ROC-AUC** – ability of the model to rank probabilities correctly
- c. The results for each model are stored in a structured format, typically a **dictionary**.

This makes it easy to collect, organize, and compare results across multiple models.

2. Final Performance Leaderboard

- a. Once all models are evaluated, their results are **combined into a single table** (commonly a **Pandas DataFrame**) to create a leaderboard.
- b. The leaderboard is usually **sorted by F1-Score**, rather than accuracy, for the following reasons:
 - i. The dataset is **imbalanced**, meaning the number of defaulters is much smaller than non-defaulters.
 - ii. F1-Score **balances detection of defaulters (recall)** with the cost of false alarms (precision), giving a fairer assessment of overall model effectiveness.
- c. The leaderboard can be **visualized using a bar chart**, showing the F1-Score or other metrics for each model. This visualization helps quickly identify the top-performing models and understand the trade-offs between metrics.

3. Interpretation of the Leaderboard

- a. Provides a **clear ranking of models** based on their ability to predict defaulters correctly while minimizing errors.
- b. Highlights which models strike the **best trade-offs** between catching defaulters and avoiding false positives.
- c. Supports informed decision-making on **which model to deploy** in production for credit risk prediction.

Summary: This structured approach ensures **fair comparison, transparent evaluation, and actionable insights** into model performance, especially in imbalanced datasets where F1-Score is the most informative metric.

11. Results

After evaluating all models, the performance metrics were collected and summarized in a **ranked table** to identify the best-performing models for credit default prediction.

Ranked Model Performance Table

Rank	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
1	Random Forest	0.746	0.447	0.622	0.520	0.764
2	Gradient Boosting	0.719	0.415	0.658	0.509	0.761
3	K-Nearest Neighbors (KNN)	0.633	0.340	0.705	0.459	0.729

4	Logistic Regression	0.566	0.298	0.710	0.420	0.712
5	Naive Bayes	0.292	0.233	0.959	0.375	0.711

Detailed Interpretation

1. Random Forest

- Performance:** Best overall performance across most metrics.
- Strengths:** Balanced **recall (0.622)** and **precision (0.447)**, meaning it effectively detects defaulters while minimizing false alarms.
- Conclusion:** Offers the best trade-off for practical deployment.

2. Gradient Boosting

- Performance:** Slightly lower precision (0.415) but similar recall (0.658) compared to Random Forest.
- Interpretation:** Slightly more false positives but still reliable in catching actual defaulters.

3. K-Nearest Neighbors (KNN)

- Performance:** High recall (0.705) but low precision (0.340).
- Interpretation:** The model detects many defaulters, but at the cost of many **false positives**, predicting non-defaulters as defaulters.

4. Logistic Regression

- Performance:** Good recall (0.710) but poor precision (0.298).
- Interpretation:** It identifies many defaulters but produces a large number of false alarms. Struggles with **non-linear relationships** in the data.

5. Naive Bayes

- Performance:** Extremely high recall (0.959) but very low accuracy (0.292).
- Interpretation:** Predicts almost everyone as a defaulter. While it rarely misses actual defaulters, the predictions are mostly unusable in practice due to poor precision.

Trade-Off Analysis

- **High Recall:**
 - Pros: Captures more actual defaulters.
 - Cons: Increases **false positives**, leading to unnecessary alerts.
- **High Precision:**
 - Pros: Reduces false alarms.
 - Cons: May miss actual defaulters.
- **Best Trade-Off:**

- Random Forest balances recall and precision, making it the most practical choice for deployment in credit default prediction.

Summary: The leaderboard highlights that while some models (like Naive Bayes or KNN) catch most defaulters, they do so at the cost of too many false positives. Random Forest provides the optimal balance, ensuring both effective detection of defaulters and manageable false alarm rates.

12. Visualizations

Visualizations are an essential part of model evaluation because they **help communicate performance** to both technical and non-technical stakeholders. They make it easier to understand not just the numbers, but the **types of errors** and the relative performance of models.

1. Confusion Matrices

- a. A **confusion matrix** is generated for each model to show:
 - i. **True Positives (TP):** Correctly predicted defaulters
 - ii. **True Negatives (TN):** Correctly predicted non-defaulters
 - iii. **False Positives (FP):** Non-defaulters incorrectly predicted as defaulters
 - iv. **False Negatives (FN):** Defaulters incorrectly predicted as non-defaulters
- b. **Purpose:** Highlights where each model makes mistakes, showing which models tend to over-predict defaults and which may miss actual defaulters.

2. Bar Chart of F1-Scores

- a. A bar chart is plotted with models on the x-axis and their **F1-Scores** on the y-axis.
- b. **Purpose:** Provides a **clear visual ranking** of models, making it immediately obvious which models achieve the best balance between precision and recall.

3. Why Visualizations Are Important

- a. Numerical metrics alone may be difficult for stakeholders to interpret.
- b. Visual aids make it easier to **compare models**, understand trade-offs, and communicate decisions.
- c. They help in **decision-making**, especially when selecting a model for deployment.

13. Conclusion

After evaluating multiple models and analyzing their performance, several key insights emerge regarding **feature engineering, pipeline design, and model selection** for credit default prediction:

1. Feature Engineering

- a. Carefully designed features such as **trends, ratios, volatility, and utilization** significantly enhance the model's predictive power.
- b. These features help the model capture meaningful patterns in customer behavior, improving its ability to distinguish between defaulters and non-defaulters.

2. Pipeline Components

- a. **RobustScaler**: Scales features while minimizing the impact of outliers, ensuring model stability.
- b. **PCA (Principal Component Analysis)**: Reduces dimensionality, removes redundant features, and improves computational efficiency.
- c. **SMOTE (Synthetic Minority Over-sampling Technique)**: Corrects class imbalance by generating synthetic samples of the minority class (defaulters), preventing the model from being biased towards non-defaulters.
- d. **Impact**: Together, these components make the pipeline **reproducible, efficient, and robust**, improving overall model performance.

3. Model Selection

- a. **Random Forest** emerged as the best-performing model based on **F1-Score**, balancing precision and recall effectively.
- b. This indicates that it can detect a significant portion of defaulters while keeping false positives at a manageable level.

4. Threshold Tuning

- a. Adjusting the decision threshold allows further optimization of **recall**, ensuring that more defaulters are correctly identified.
- b. This is particularly important in finance, where missing a defaulter can have significant monetary consequences.

14. Future Work

While the current study demonstrates strong results with Random Forest and other models, several avenues exist for further improving predictive performance and real-world applicability:

1. Hyperparameter Tuning for All Models

- a. Fine-tuning model parameters (e.g., number of trees, maximum depth, learning rate) can significantly improve performance metrics.
- b. Using techniques like **Grid Search** or **Random Search** ensures that each model is optimized for the dataset, potentially improving recall, precision, and F1-Score.

2. Advanced Ensemble Methods

- a. Models such as **XGBoost** and **LightGBM** are highly efficient gradient boosting algorithms.
- b. These methods often outperform traditional models like Random Forest due to better handling of feature interactions and faster training times.

3. Model Explainability Using SHAP

- a. SHAP (SHapley Additive exPlanations) can be used to **explain individual predictions**, showing which features contribute most to a prediction.
- b. This is particularly important in finance, where stakeholders need transparent and interpretable decisions for compliance and trust.

4. Real-Time Credit Scoring System Implementation

- a. Deploying the best-performing model in a **real-time environment** enables instant risk assessment for new customers.
- b. Incorporating pipeline components, threshold tuning, and monitoring ensures that predictions remain accurate and actionable.